Greedy algorithm

Problem 2 Suppose you are the system administrator and you have a really fancy computer at your command and a lot of people want to use it. They come and give you their job. You have a slot of 24 hours and you have to schedule them. Each person also give you their slot eg sombody comes and give you the slot say 3am to 5am. Sombay say 2PM to 7PM. Their interval could vary. Each person gives ~~you~~ exactly ~~exactly~~ one interval. People don't want shorter intervals they wanted the machine to use for their entire duration which they have asked for

You job is to schedule these intervals on the machine. S.t.

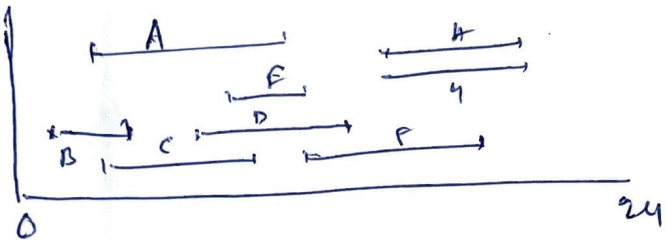
1. ~~only~~ each time only on person can work on the machine
2. maximum number of people would get to use the machine in the 24 hrs slot

Your policy should not be dictated by any other fact like the amount of ~~money~~ money they are willing to give you.

Here is an abstract sort of version of the problem

input: Set of intervals

output: subset of nonoverlapping intervals of maximum size



Say for now

By size of the subset we mean the no of intervals in the subset. Our goal is not to utilize the whole line but maximize the number of intervals.

well how do we solve this?

The so called greedy technique says build the solution piece by piece.

So we need to pick build the optimum solution interval by interval.

So which interval you first pick and thinks that it would lie in the optimum solution.

what does you intuition say?

So you want pick intervals so that a large no of intervals can be accomodated.

So by intuition I think first ~~that the~~ idea that sort of ~~thing~~ strikes you is ~~that~~ if I pick smaller intervals ie interval of small size may be I can accomodate large no of them.

Algorithm 1

- Pick interval of smallest span (length) this seems ~~to be a~~ a reasonable ~~greedy~~ algo. this is what perhaps the intuition suggest but this ~~algorithm~~ intuition fails. It is very easy to construct examples where the algorithm fails

b-a is length

the algorithm says pick the interval a but once you pick a you can't pick b and c. Clearly optimum is consist of b & c

So the first thing that we try fails. So ~~the~~ picking the interval of smallest length fail. Well let us think more. what else could wrong there?

If you look at the example, ~~even~~ though the interval ~~a~~ a is small it overlaps with two of them.

So may be the right thing to do is to pick an interval that overlap with smallest no of intervals.

Algo2

— pick the interval that ~~with~~ overlaps with smallest no of intervals.

So, The first algorithm fails, you construct an example when the algorithm fails and there is another algorithm which atleast works on that example. It seems fairly reasonable where you pick interval that ~~which~~ overlaps ~~the~~ with smallest no of intervals. So when I pick the interval we throw away the smallest no of intervals and we recurse on the remaining intervals. The Algorithm say that once I pick an interval, ~~I throw away exactly~~

every interval that overlaps with this interval I am not going to pick. I throw those away and I recurse. This should looks ~~fairly reasonable~~ very familiar to the previous problem. There you pick a vertex, throw away its neighbors and go forward.

Here you pick an interval throw away the interval with which this overlap and you proceed forward. Well they are extremely closely related.

Infact we could ~~form~~ a graph where ① there is a vertex for ~~every~~ every interval. ② there is an edge ~~between~~ between two vertices if their corresponding interval overlaps the all you are doing for this problem is picking an independent set of maximum size in this graph.

So like in the previous case why not pick an vertex of smallest degree. The minimum degree vertex is nothing but the interval with smallest no of overlaps.

So you pick a vertex of minimum degree

which actually means you are picking an interval with smallest no of overlaps. throw away its neighbors and recurse. So that's the algorithm

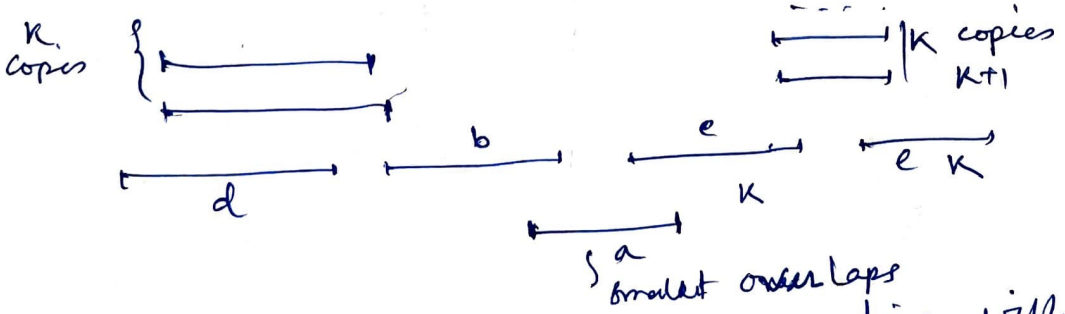
Now the question is does this work?

Well it worked in the previous case,

Unfortunately in this case it does not work.

Here is the example

So this is a bit more complicated. The ~~problem~~ example ~~that~~ which I have to construct now is, for which the algorithm does not work is a ~~&~~ bit more complicated.



each of the k copies of the interval overlaps with K-1 plus 2 intervals so K+1 interval is what this thing overlaps with

right side is mirror image of the left side

The left and right side intervals has large overlaps. The smallest overlaps is in the middle portion. Infact the interval a is the interval with smallest overlaps infact its is 2.

The algorithm says pick the interval a. and recurse on the remaing after removing its neighbor.

Lets see what happens if we pick a, The interval b and c should be thrown ait as they overlaps with a. So now we are left with left side and the right side. It is obvious from the picture that we can only pick one from each side.

So the size of the solution that we have constructed is three (3)

So if we look at the figure carefully we can see that the optimum has the value 3.4. consisting of b.c, d, e. This intervals do not overlaps with each other. So we see that our algorithm produced as output 3 intervals whereas the optimum has the value 4. So this algorithm does not work though the ideas initially looked promising. So what we have seen that the algorithms for roughly the same kind of problem worked for one and do not worked for the other. And carry up with example for which it does not work becomes harder and harder..

So You can come up with really complicated problem algorithm for which it may not work but to prove that it does not work, you have to come up with one kind of a complicated counter example.

So what do we do? What is the algorithm that works. Does there even

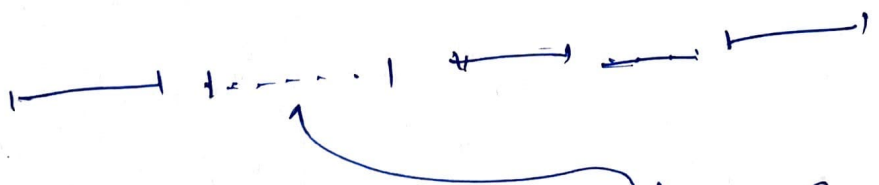
exist and algorithm that works.

So let us look at the optimum. So optimum look like this

Any optimum solution must be a set of intervals that does not overlap with each other.

Now let us apply the exchange trick. Can I add an interval here. Well ~~certainly~~ certainly if we have an space

Supposing how the solution looks like



I can certainly add one here, Supposing these space are all filled up. Can I add (I may not be able to add) and remove one? is that possible?

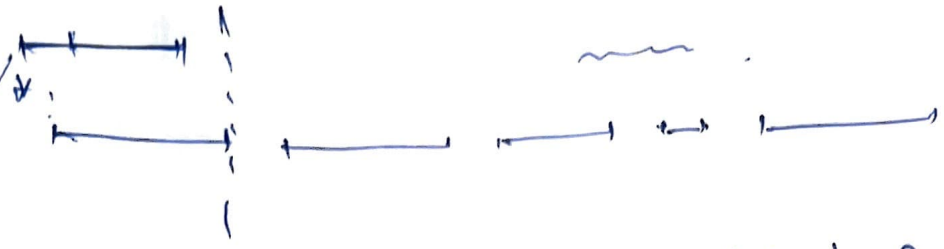
Supposing we add an interval I as follow

Now this could ~~overlap~~ overlap with more than one intervals. In that case adding the new one means no sense. There is no exchange that sort of keeps you going.

If I ~~change~~ exchange this, I have to remove more than one interval and the size of the optimum falls. Now the question is, is ~~there~~ ~~little~~ there any interval that I can always add I to this optimum solution? and ~~that~~ may ~~be~~ I be more at most one?

This require a ~~little~~ little bit of thought and I tell you that there is an interval that I can always add. and the interval is the one that ends first.

Lets see what this means.



By the interval that ends finest I mean it has to end before the ~~[illegible]~~ interval ~~with the the [illegible]~~

interval that ends first.

Now you can see that I can add this and remove atmost one interval ie the first interval ~~in the solution~~ This will certainly not intersect with the new. certainly not intersect ~~the~~ with the second interval. Because the starting time of the second interval comes after ending time of the first interval. and ending time of the new interval is either same as end time of the first interval or ~~less~~ befor that.

So we can always add the interval that ends fasst. and remove the ~~one~~

first interval of the earliest chosen set of intervals if need be. If the new interval also ends before the start of the first interval then we are also actually increasing the size of the solution.

So exchange trick plus some intuition tells us that we can always add the interval that ends first. So give me any solution I can add an interval that ends first may be & remove the first interval. Certainly the size of the solution does not go down

So this idea leads us to design the algorithm which actually works

Alg 3: Add intervals that ends first, remove the interval that overlap with this and

Recurse

This is the algorithm & gives us what we want.

Instead of recursing we can sort the intervals with their ending time. and pick the interval that ends first, throw out the interval that this overlaps and then continue. You can write this as an iterative procedure

This also work. Why does this work? It is this exchange trick that makes it work and we can prove it by Induction.

First thing to prove would be to show that there is an optimal solution which contains the interval that ends first.